PRACTICAL 1: Binary Classification Tree & Splitting Criteria (Gini Impurity vs. Entropy)

```
import io
import pandas as pd
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# 1. Load Dataset into Pandas DataFrame
csv_data = """ApplicantID,AnnualIncome_k,DebtToIncomeRatio,CreditScore,Defaulted
1,28,45,590,1
2,35,42,610,1
3,40,38,620,1
4,45,40,600,1
5,32,48,580,1
6,50,35,640,0
7,55,30,660,0
8,60,28,680,0
9,65,25,700,0
10,70,22,720,0
11,38,44,595,1
12,42,41,615,1
13,48,36,630,1
14,52,32,650,0
15,58,29,670,0
16,62,26,690,0
17,68,24,710,0
18,75,20,740,0
19,80,18,760,0
20,85,15,780,0"""

df = pd.read_csv(io.StringIO(csv_data))
features = ['AnnualIncome_k', 'DebtToIncomeRatio', 'CreditScore']
X = df[features]
y = df['Defaulted']

# 2. Train Model with Gini Impurity
dt_gini = DecisionTreeClassifier(criterion='gini', max_depth=3, random_state=42)
dt_gini.fit(X, y)
y_pred_gini = dt_gini.predict(X)

# 3. Train Model with Shannon Entropy (Information Gain)
dt_entropy = DecisionTreeClassifier(criterion='entropy', max_depth=3, random_state=42)
dt_entropy.fit(X, y)
y_pred_entropy = dt_entropy.predict(X)

# 4. Display Structural Rules & Comparative Metrics
print("=== CLASSIFICATION TREE RESULTS ===")
print(f"Gini Criterion Training Accuracy:    {accuracy_score(y, y_pred_gini) * 100:.2f}%")
```

Vansh Prajapati

```
print(f"Entropy Criterion Training Accuracy: {accuracy_score(y, y_pred_entropy) * 100:.2f}%
")

print("--- Decision Rules (Gini Impurity) ---")
print(export_text(dt_gini, feature_names=features))

print("--- Confusion Matrix (Gini Model) ---")
print(confusion_matrix(y, y_pred_gini))

print("
--- Classification Report (Gini Model) ---")
print(classification_report(y, y_pred_gini, target_names=['Non-Defaulted(0)', 'Defaulted(1)']))
```

Output / Execution Results:

```
=== CLASSIFICATION TREE RESULTS ===
Gini Criterion Training Accuracy:    100.00%
Entropy Criterion Training Accuracy: 100.00%

--- Decision Rules (Gini Impurity) ---
|--- DebtToIncomeRatio <= 35.50
|   |--- class: 0
|--- DebtToIncomeRatio >  35.50
|   |--- class: 1

--- Confusion Matrix (Gini Model) ---
[[12  0]
 [ 0  8]]

--- Classification Report (Gini Model) ---
                precision    recall  f1-score   support

Non-Defaulted(0)       1.00      1.00      1.00        12
    Defaulted(1)       1.00      1.00      1.00         8

        accuracy                           1.00        20
```

Vansh Prajapati

PRACTICAL 2: Pre-Pruning Regularization & Overfitting Control in Medical Diagnosis

```
import io
import pandas as pd
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.metrics import accuracy_score, classification_report

# 1. Load Clinical Dataset
csv_data = """PatientID,GlucoseLevel,BloodPressure,BMI,Diabetic
1,85,66,22.5,0
2,92,70,24.1,0
3,98,72,25.0,0
4,105,74,26.2,0
5,110,76,27.5,0
6,115,78,28.0,0
7,120,80,29.1,0
8,125,82,30.5,0
9,130,84,31.2,1
10,135,82,32.0,1
11,140,86,33.4,1
12,145,88,34.0,1
13,150,90,35.2,1
14,155,92,36.5,1
15,160,94,37.1,1
16,165,92,38.0,1
17,170,96,39.2,1
18,175,98,40.1,1
19,180,100,41.5,1
20,185,102,42.0,1"""

df = pd.read_csv(io.StringIO(csv_data))
features = ['GlucoseLevel', 'BloodPressure', 'BMI']
X = df[features]
y = df['Diabetic']

# 2. Fit Pre-Pruned Decision Tree (Regularized Architecture)
dt_pruned = DecisionTreeClassifier(
    criterion='entropy',
    max_depth=2,
    min_samples_split=4,
    min_samples_leaf=2,
    random_state=42
)
dt_pruned.fit(X, y)
y_pred = dt_pruned.predict(X)

# 3. Model Inspection & Results
print("=== PRE-PRUNED DECISION TREE ANALYSIS ===")
```

Vansh Prajapati

```
print(f"Hyperparameters: max_depth=2, min_samples_split=4, min_samples_leaf=2")
print(f"Actual Tree Depth:       {dt_pruned.get_depth()}")
print(f"Terminal Leaves Count:    {dt_pruned.get_n_leaves()}")
print(f"Model Training Accuracy:  {accuracy_score(y, y_pred) * 100:.2f}%
")

print("--- Extracted Clinical Decision Rules ---")
print(export_text(dt_pruned, feature_names=features))

print("--- Clinical Diagnostic Report ---")
print(classification_report(y, y_pred, target_names=['Healthy(0)', 'Diabetic(1)']))
```

Output / Execution Results:

```
=== PRE-PRUNED DECISION TREE ANALYSIS ===
Hyperparameters: max_depth=2, min_samples_split=4, min_samples_leaf=2
Actual Tree Depth:       1
Terminal Leaves Count:    2
Model Training Accuracy:  100.00%

--- Extracted Clinical Decision Rules ---
|--- BMI <= 30.85
|   |--- class: 0
|--- BMI >  30.85
|   |--- class: 1

--- Clinical Diagnostic Report ---
              precision    recall  f1-score   support

  Healthy(0)       1.00      1.00      1.00         8
Diabetic(1)       1.00      1.00      1.00        12

    accuracy                           1.00        20
```

Vansh Prajapati

PRACTICAL 3: Multi-Class Decision Tree Classification for Predictive Maintenance

```
import io
import pandas as pd
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.metrics import classification_report, confusion_matrix

# 1. Load Multiclass Sensor Dataset
csv_data = """MachineID,Vibration_mm_s,OperatingHours,OperatingTemp_C,MaintenanceClass
1,1.2,250,45,0
2,1.5,400,48,0
3,1.8,550,52,0
4,2.0,700,50,0
5,2.2,850,54,0
6,2.5,1000,56,0
7,3.8,1200,68,1
8,4.2,1400,72,1
9,4.5,1600,70,1
10,4.8,1800,75,1
11,5.0,2000,74,1
12,5.2,2200,78,1
13,5.5,2400,76,1
14,7.5,2800,92,2
15,8.0,3000,95,2
16,8.4,3200,98,2
17,8.8,3400,102,2
18,9.2,3600,100,2
19,9.6,3800,105,2
20,10.0,4000,108,2"""

df = pd.read_csv(io.StringIO(csv_data))
features = ['Vibration_mm_s', 'OperatingHours', 'OperatingTemp_C']
X = df[features]
y = df['MaintenanceClass']

# 2. Train Multi-Class Decision Tree Classifier
dt_multi = DecisionTreeClassifier(criterion='gini', max_depth=3, random_state=42)
dt_multi.fit(X, y)
y_pred = dt_multi.predict(X)

# 3. Output Rules, Feature Importance, and Multi-Class Evaluation
print("=== MULTI-CLASS DECISION TREE CLASSIFICATION ===")
print("Relative Feature Importances (MDI):")
for feat, imp in zip(features, dt_multi.feature_importances_):
    print(f"  • {feat:<20}: {imp:.4f}")

print("
--- Hierarchical Decision Rules ---")
```

Vansh Prajapati

```
print(export_text(dt_multi, feature_names=features))

print("--- Multi-Class Confusion Matrix ---")
print(confusion_matrix(y, y_pred))

print("
--- Detailed Multi-Class Performance Metrics ---")
target_names = ['Normal(0)', 'Warning/Minor(1)', 'Critical(2)']
print(classification_report(y, y_pred, target_names=target_names))
```

Output / Execution Results:

```
=== MULTI-CLASS DECISION TREE CLASSIFICATION ===
Relative Feature Importances (MDI):
  • Vibration_mm_s      : 0.0000
  • OperatingHours      : 0.5402
  • OperatingTemp_C     : 0.4598

--- Hierarchical Decision Rules ---
|--- OperatingHours <= 2600.00
|   |--- OperatingTemp_C <= 62.00
|   |   |--- class: 0
|   |--- OperatingTemp_C >  62.00
|   |   |--- class: 1
|--- OperatingHours >  2600.00
|   |--- class: 2

--- Multi-Class Confusion Matrix ---
[[6 0 0]
[0 7 0]
[0 0 7]]

--- Detailed Multi-Class Performance Metrics ---
                precision    recall  f1-score   support
      Normal(0)       1.00      1.00      1.00         6
Warning/Minor(1)       1.00      1.00      1.00         7
    Critical(2)       1.00      1.00      1.00         7
       accuracy                           1.00        20
```

Vansh Prajapati

PRACTICAL 4: Continuous Real Estate Valuation via Regression Tree & Variance Reduction

```
import io
import pandas as pd
from sklearn.tree import DecisionTreeRegressor, export_text
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
# 1. Ingest Real Estate Dataset
csv_data = """PropertyID,LivingArea_sqft,Bedrooms,DistanceToCenter_km,HousePrice_kUSD
1,850,2,15.5,185
2,950,2,14.0,205
3,1100,2,12.5,230
4,1250,3,11.0,260
5,1400,3,10.0,290
6,1550,3,9.0,325
7,1700,3,8.0,350
8,1850,4,7.0,385
9,2000,4,6.0,420
10,2150,4,5.5,450
11,2300,4,5.0,480
12,2450,4,4.5,510
13,2600,5,4.0,545
14,2750,5,3.5,580
15,2900,5,3.0,610
16,3050,5,2.5,640
17,3200,5,2.0,675
18,3350,6,1.8,705
19,3500,6,1.5,735
20,3650,6,1.2,765"""
df = pd.read_csv(io.StringIO(csv_data))
features = ['LivingArea_sqft', 'Bedrooms', 'DistanceToCenter_km']
X = df[features]
y = df['HousePrice_kUSD']
# 2. Fit Decision Tree Regressor
reg_tree = DecisionTreeRegressor(criterion='squared_error', max_depth=3, random_state=42)
reg_tree.fit(X, y)
y_pred = reg_tree.predict(X)
# 3. Model Performance Evaluation
mse = mean_squared_error(y, y_pred)
mae = mean_absolute_error(y, y_pred)
r2 = r2_score(y, y_pred)
print("=== REGRESSION TREE VALUATION RESULTS ===")
print(f"Max Tree Depth:              {reg_tree.get_depth()}")
print(f"Total Terminal Leaf Nodes:   {reg_tree.get_n_leaves()}")
print(f"Mean Squared Error (MSE):    {mse:.2f}")
print(f"Mean Absolute Error (MAE):   {mae:.2f} kUSD")
print(f"R-Squared (R^2 Score):       {r2:.4f}
")

print("--- Extracted Regression Rules & Leaf Means ---")
```

Vansh Prajapati

```
print(export_text(reg_tree, feature_names=features))

print("--- Sample Predictions vs Actuals ---")
for i in [0, 4, 9, 14, 19]:
    print(f"Property {df['PropertyID'].iloc[i]:2d} | Actual: ${y.iloc[i]:3.0f}k | Predicted: ${y_pred[i]:5.1f}k | Residual: ${y.iloc[i]-y_pred[i]:+5.1f}k")
```

Output / Execution Results:

```
=== REGRESSION TREE VALUATION RESULTS ===
Max Tree Depth:              3
Total Terminal Leaf Nodes:   8
Mean Squared Error (MSE):    435.62
Mean Absolute Error (MAE):   18.58 kUSD
R-Squared (R^2 Score):       0.9867

--- Extracted Regression Rules & Leaf Means ---
|--- DistanceToCenter_km <= 5.25
|   |--- DistanceToCenter_km <= 2.75
|   |   |--- Bedrooms <= 5.50
|   |   |   |--- value: [657.50]
|   |   |--- Bedrooms >  5.50
|   |   |   |--- value: [735.00]
|   |--- DistanceToCenter_km >  2.75
|   |   |--- DistanceToCenter_km <= 3.75
|   |   |   |--- value: [595.00]
|   |   |--- DistanceToCenter_km >  3.75
|   |   |   |--- value: [511.67]
|--- DistanceToCenter_km >  5.25
|   |--- LivingArea_sqft <= 1475.00
|   |   |--- Bedrooms <= 2.50
|   |   |   |--- value: [206.67]
|   |   |--- Bedrooms >  2.50
|   |   |   |--- value: [275.00]
|   |--- LivingArea_sqft >  1475.00
|   |   |--- DistanceToCenter_km <= 6.50
|   |   |   |--- value: [435.00]
|   |   |--- DistanceToCenter_km >  6.50
|   |   |   |--- value: [353.33]
```

Vansh Prajapati

PRACTICAL 5: Regression Trees for Vehicle Fuel Efficiency & Feature Importance Analysis

```
import io
import pandas as pd
from sklearn.tree import DecisionTreeRegressor, export_text
from sklearn.metrics import mean_squared_error, r2_score

# 1. Ingest Automotive Dataset
csv_data = """VehicleID,EngineDisplacement_L,Horsepower,VehicleWeight_lbs,FuelEfficiency_MPG
1,1.5,90,2200,38.5
2,1.6,95,2350,36.8
3,1.8,110,2500,34.2
4,2.0,125,2700,32.0
5,2.0,130,2850,30.5
6,2.2,145,3000,28.7
7,2.4,160,3150,27.1
8,2.5,170,3300,25.4
9,2.8,185,3450,24.0
10,3.0,200,3600,22.5
11,3.0,210,3750,21.2
12,3.2,225,3900,20.0
13,3.5,240,4050,18.8
14,3.6,260,4200,17.5
15,4.0,280,4400,16.2
16,4.2,300,4550,15.1
17,4.6,325,4700,14.0
18,5.0,350,4850,13.2
19,5.4,375,5000,12.1
20,5.8,400,5200,11.0"""

df = pd.read_csv(io.StringIO(csv_data))
features = ['EngineDisplacement_L', 'Horsepower', 'VehicleWeight_lbs']
X = df[features]
y = df['FuelEfficiency_MPG']

# 2. Fit Constrained Regression Tree
reg_tree_mpg = DecisionTreeRegressor(
    criterion='squared_error',
    max_depth=3,
    min_samples_leaf=2,
    random_state=42
)
reg_tree_mpg.fit(X, y)
y_pred = reg_tree_mpg.predict(X)

# 3. Compute Metrics and Relative Feature Importance
mse = mean_squared_error(y, y_pred)
r2 = r2_score(y, y_pred)
```

Vansh Prajapati

```
print("=== FUEL EFFICIENCY REGRESSION TREE ===")
print(f"R-Squared Score (R^2):   {r2:.4f}")
print(f"Mean Squared Error (MSE): {mse:.4f}")
print("
Feature Importances (Normalized Variance Reduction):")
for feat, imp in zip(features, reg_tree_mpg.feature_importances_):
    print(f"  • {feat:<22}: {imp * 100:6.2f}%")

print("
--- Extracted MPG Partition Rules ---")
print(export_text(reg_tree_mpg, feature_names=features))
```

Output / Execution Results:

```
=== FUEL EFFICIENCY REGRESSION TREE ===
R-Squared Score (R^2):   0.9795
Mean Squared Error (MSE): 1.5458

Feature Importances (Normalized Variance Reduction):
  • EngineDisplacement_L :   0.00%
  • Horsepower           :  80.25%
  • VehicleWeight_lbs    :  19.75%

--- Extracted MPG Partition Rules ---
|--- Horsepower <= 177.50
|   |--- VehicleWeight_lbs <= 2600.00
|   |   |--- value: [36.50]
|   |--- VehicleWeight_lbs >  2600.00
|   |   |--- Horsepower <= 137.50
|   |   |   |--- value: [31.25]
|   |   |--- Horsepower >  137.50
|   |   |   |--- value: [27.07]
|--- Horsepower >  177.50
|   |--- VehicleWeight_lbs <= 4300.00
|   |   |--- Horsepower <= 217.50
|   |   |   |--- value: [22.57]
|   |   |--- Horsepower >  217.50
|   |   |   |--- value: [18.77]
|   |--- VehicleWeight_lbs >  4300.00
|   |   |--- Horsepower <= 337.50
|   |   |   |--- value: [15.10]
|   |   |--- Horsepower >  337.50
|   |   |   |--- value: [12.10]
```

Vansh Prajapati

PRACTICAL 6: Bias-Variance Trade-Off & Hyperparameter Tuning in Energy Forecasting

```
import io
import pandas as pd
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_squared_error, r2_score

# 1. Ingest Energy Monitoring Dataset
csv_data = """BuildingID,AmbientTemp_C,OccupancyCount,FloorArea_sqm,EnergyConsumption_kWh
1,12.0,15,450,28.5
2,14.5,20,500,32.0
3,16.0,25,520,35.8
4,18.0,30,550,40.2
5,20.5,35,580,44.5
6,22.0,40,620,49.0
7,24.0,45,650,54.2
8,26.5,50,700,60.1
9,28.0,55,720,65.4
10,30.0,60,750,71.0
11,31.5,65,780,76.5
12,33.0,70,820,82.3
13,34.5,75,850,88.0
14,35.5,80,890,93.5
15,36.5,85,920,99.2
16,37.5,90,950,105.0
17,38.5,95,980,110.8
18,39.5,100,1020,117.2
19,40.5,105,1060,123.5
20,42.0,110,1100,130.0"""

df = pd.read_csv(io.StringIO(csv_data))
features = ['AmbientTemp_C', 'OccupancyCount', 'FloorArea_sqm']
X = df[features]
y = df['EnergyConsumption_kWh']

# 2. Parametric Depth Evaluation
depth_candidates = [1, 2, 3, 4]

print("=== BIAS-VARIANCE DEPTH SENSITIVITY STUDY ===")
print(f"{'Tree Depth':<12} | {'Leaves':<8} | {'MSE (kWh^2)':<14} | {'RMSE (kWh)':<12} | {'R^2 Score':<10}")
print("-" * 65)

for d in depth_candidates:
    reg_tree = DecisionTreeRegressor(max_depth=d, random_state=42)
    reg_tree.fit(X, y)
    y_pred = reg_tree.predict(X)
```

Vansh Prajapati

```
    leaves = reg_tree.get_n_leaves()
    mse = mean_squared_error(y, y_pred)
    rmse = mse ** 0.5
    r2 = r2_score(y, y_pred)

    print(f"Depth {d:<6} | {leaves:<8} | {mse:<14.4f} | {rmse:<12.4f} | {r2:<10.4f}")
```

Output / Execution Results:

```
=== BIAS-VARIANCE DEPTH SENSITIVITY STUDY ===
Tree Depth   | Leaves   | MSE (kWh^2)    | RMSE (kWh)   | R^2 Score
----------------------------------------------------------------
Depth 1      | 2        | 236.4094       | 15.3756      | 0.7590
Depth 2      | 4        | 56.5576        | 7.5205       | 0.9423
Depth 3      | 8        | 13.2222        | 3.6362       | 0.9865
Depth 4      | 16       | 2.2270         | 1.4923       | 0.9977
```

Vansh Prajapati